

Final Exam Topics

22. Functional programming

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

TODO: Highly incomplete!

Functional programming

Characteristics of functional programming languages: characterization and comparison of lazy and eager evaluation strategies, referential transparency, static typing, Currying principle, offside (layout) rule, basic types, conversions, defining and typing functions, pattern matching, guards, case distinction, recursion, local definitions, higher-order functions, anonymous functions, function composition, list comprehensions, type classes, parametric and ad-hoc polymorphism, type synonyms, defining algebraic data types.

1 Lazy and eager evaluation

The expression evaluation of functional languages can be lazy or eager (strict). During lazy evaluation the arguments of functions are evaluated only if they are absolutely needed. Clean uses such lazy evaluation, while Erlang belongs rather among the strict languages. Eager evaluation evaluates the expressions in every case, namely as soon as it is possible. Lazy evaluation would interpret the expression `inc 4+1` first as the expression $(4+1)+1$, while the eager system would immediately bring it to the form $5 + 1$.

2 Referential transparency

The value of expressions does not depend on where they occur in the text of the program, that is, wherever we place the same expression, its result remains the same. This is most true for purely functional languages, where the functions have no side effect, so during their evaluation they do not change the given expression either.

3 Static type system

In static type systems giving the declarations is not mandatory, but it is a requirement that the compiler be able to infer the most general type of the expression in every case. The basis of the static system is the Hindley-Milner polymorphic type system.

Of course, the omissibility of giving the types does not mean that they are not present in the given language. There certainly are types; only the type inference system and the compiler take care of their handling, as well as of the correct evaluation of the expressions.

4 Currying principle

The Curry method is partial function application, which means that the return value of multi-parameter functions can be a function and the remaining parameters. Based on this, every function can be regarded as a single-variable function. If the function has two variables, then we regard only one of them as the parameter belonging to the function. Giving the parameter creates a new single-variable function, which we can apply to the second variable. The method also works for several variables. In Erlang and F# there is no such language element, but the possibility is given. For its creation we can use a function expression.

5 Offside (layout) rule

Changing the width of the left margin is suitable for identifying groups of related expressions and for limiting the scope of declarations.

6 Basic types

TODO

7 Pattern matching

Pattern matching, applied to functions, means that the function can have several branches, and to the individual branches (clauses) we can assign different patterns of the formal parameters. When the function is called, the branch that runs is always the one to whose formal parameters the actual parameters given at the call match. In OO languages we call such functions overload functions. Pattern matching can be applied for matching variables and lists, as well as for choosing the branches of branchings.

8 Higher-order functions

In functional languages we can pass lambda expressions, that is, specially described functions, in the parameter list of other functions. Of course, it is not the function call that gets among the parameters, but the prototype of the function.

Numerous programming languages give the possibility to bind lambda functions to variables, then later use the variables constructed this way as functions. We can also place lambda expressions in list expressions. Parameterization with a function helps us create completely general functions or programs, which is especially useful when making server applications.

9 List comprehensions

Besides the various language constructs, in functional languages we also find numerous special data types, such as the ordered n-tuple, or by its more familiar name the tuple, as well as list comprehensions and the list generators that can be constructed from them. The list data type is based on the Zermelo-Fraenkel set comprehension. It contains a generator, which describes the condition of the elements' membership in the list, as well as how and in what number the individual elements of the list have to be produced. With this technology we are in principle able to describe even an infinite list in the given programming language, since we describe not the elements of the list, but the definition of the list.